CodeDNA: An In-Source Annotation Standard for Deterministic Navigation of Multi-File Codebases by LLM Agents

Fabrizio Corpora

Independent Research · *fabrizio.corpora@gmail.com*

Zenodo preprint · March 2026 · doi:10.5281/zenodo.19109240

Abstract

LLM agents that autonomously edit production codebases suffer from *context blindness*: they modify a function without awareness of cross-file dependencies, producing silent bugs and broken cascades. Existing mitigations — RAG, project-level rules files (CLAUDE.md, AGENTS.md), or long-context injection — impose retrieval latency, maintenance overhead, or infrastructure cost. Recent work (ETH Zurich, 2026) shows that LLM-generated context files can *degrade* task success by 3% and increase inference costs by over 20%. We present **CodeDNA**, an in-source annotation standard that embeds dependency graphs, public contracts, agent constraints, and semantic naming conventions directly into source files. The standard is named after its biological parallel: just as DNA encodes the complete blueprint of an organism in every cell, CodeDNA ensures that every code fragment carries sufficient architectural context for an agent to act correctly in isolation. We evaluate CodeDNA with three benchmarks: (1) an 11-file codebase — **−25% tool calls, −33% file reads**; (2) a 105-file enterprise platform — **−23% tool calls, zero incorrect file identifications**; and (3) a multi-model SWE-bench evaluation across **3 LLMs** (Gemini 2.5 Flash, DeepSeek Chat, Gemini 2.5 Pro) on 5 real Django issues, 5 runs/task at T=0.1 — **CodeDNA improved file localization F1 on both completed models**, with a statistically significant gain of +13 percentage points on Gemini 2.5 Flash (60% → 72%, Wilcoxon p=0.040) and +9pp on DeepSeek Chat (50% → 60%). These results measure *navigation efficiency* (which files the agent reads), not end-to-end patch correctness, isolating the navigation bottleneck that CodeDNA specifically addresses. CodeDNA requires no build step, no external system, and zero retrieval latency.

1. Introduction

The adoption of LLM coding agents in production workflows has exposed a class of failure that single-file benchmarks routinely miss: *cross-file context blindness*. When an agent reads a function in isolation — because it operates in sliding-window mode or because the relevant contract resides in another module — it cannot observe the architectural constraints governing correct behaviour.

SWE-bench [1], the field's principal multi-file benchmark, demonstrated in 2024 that state-of-the-art models could resolve only 1.96% of real GitHub issues. The dominant failure mode was not reasoning quality but *navigation cost*: finding the right files required too many tool interactions. Subsequent work confirms the pattern: LoCoBench-Agent [2] (2025) measures tool-usage efficiency in long-context engineering workflows and finds systematic over-exploration; PRDBench [3] (2025) reports that agents spend the majority of their token budget on navigation rather than repair.

Current mitigation approaches share a structural weakness: *context lives outside the code*. CLAUDE.md and CursorRules are session-level files invisible to agents in sliding-window mode. RAG [6] systems introduce retrieval latency and require embedding infrastructure. A 2026 ETH Zurich study [4] found that LLM-generated context files (AGENTS.md) degraded task success by 3% and increased costs by over 20%.

We propose a complementary approach: embed context *inside the code*, at every level, so that any fragment an agent reads is already self-sufficient.

Contributions:

(1) The **CodeDNA Annotation Standard v0.5**: a three-level, language-agnostic in-source annotation protocol.

(2) A biological analogy that frames the standard's design principle and motivates its redundancy property.

(3) Two **live agentic benchmarks** (11 files and 105 files) using Gemini 2.5 Flash with function-calling tools on real on-disk codebases.

(4) A **multi-model SWE-bench benchmark** across 5 LLMs demonstrating broadly beneficial improvements in file localization F1, with heterogeneous gains across model families.

(5) Evidence of **−23% to −25% tool-call reduction**, zero incorrect diagnoses in custom benchmarks, and +20pp F1 improvement on the best-performing model.

(6) Open-source release of spec, benchmark code, and AI integration files.

2. Related Work

2.1 Multi-File Agent Benchmarks

SWE-bench [1] (Jimenez et al., 2024) evaluates LLMs on 2,294 real GitHub issues across 12 Python repositories. It established

that multi-file coordination is the principal bottleneck, with a 1.96% resolution rate at launch. CodeDNA addresses the *navigation cost* dimension that SWE-bench exposed.

LoCoBench-Agent [2] (November 2025) provides the closest methodological parallel to our work: it benchmarks LLM agents in long-context software engineering scenarios with 8,000 tasks and measures *tool-usage efficiency* explicitly. Our metrics (tool calls per bug, file reads, grep frequency) directly parallel LoCoBench-Agent's efficiency framing.

PRDBench [3] (October 2025, presented at AAMAS 2026) uses 50 real Python projects and an Agent-as-a-Judge paradigm to evaluate agents from Product Requirement Document to implementation. It confirms that agents over-explore when codebase structure is opaque.

HumanEval [5] (Chen et al., 2021) measures single-function synthesis from docstrings. Its *pass@k* metric is complementary: it captures output quality but not cross-file navigation, making it orthogonal rather than competing.

2.2 Context Management for LLM Agents

Retrieval-Augmented Generation [6] retrieves relevant chunks from a vector store at inference time. Applied to code, retrieval quality depends on embedding similarity, which may not align with architectural relevance and requires additional infrastructure.

Rules files (CLAUDE.md [10], CursorRules, AGENTS.md, GitHub Copilot instructions [9]) impose project-level conventions but are invisible to sliding-window agents and require manual maintenance. The ETH Zurich study [4] provides the strongest evidence against the external-file approach: even carefully crafted human-written files offer only a 4% gain, and LLM-generated ones actively degrade performance by −3% while increasing costs by +20%.

MemGPT [7] maintains compressed memory across turns. This addresses recall of prior context but not forward navigation: a history of prior reads does not tell an agent what to read *next*. Knowledge graph–based approaches (Tongyi Lingma [11], 2025–2026) use graph traversal to expose cross-module call relationships but rely on external infrastructure and dynamic analysis. **CodeDNA's key differentiator**: the navigation graph is co-located with the code itself, requiring no retrieval system and surviving in any sliding window.

2.3 Code Annotation and Documentation Standards

Documentation standards (docstrings, JSDoc, Doxygen, Literate Programming [8]) focus on human readability and API documentation generation. They do not encode cross-file navigation semantics or agent-specific constraints. Type annotation systems (PEP 484, TypeScript) encode type contracts but not architectural ones. CodeDNA differs by designing annotations as a *machine-readable navigation protocol* targeting the failure modes of LLM agents rather than human readers.

3. The CodeDNA Annotation Standard v0.5

3.1 Biological Analogy

The name *CodeDNA* captures the standard's core design principle by analogy to molecular biology. Biological DNA encodes the complete blueprint of an organism in *every cell*: each cell's nucleus contains the full genome, not just the instructions for its own specialised function. This redundancy means that any cell — regardless of its location or context — can be read by the right machinery and yield architectural information about the whole organism.

CodeDNA applies the same principle to software: every source file encodes not only its own logic but also its *position in the architectural organism* — what it depends on, what depends on it, what contracts it must honour. The three levels mirror three scales of biological encoding:

Table A. Biological DNA ↔ CodeDNA Analogy

Biological concept	CodeDNA equivalent
Full genome in every cell	L1 module header in every file: full architectural position
Gene = local function unit	L2 function docstring: local contract + cross-file deps
Codon = minimal encoding unit	L3 inline comment at call site: 1-line context at danger point
Holographic redundancy	Any 10-line extract carries enough context to navigate safely
No external lookup needed	Zero retrieval latency; context travels with the code

The holographic property — cutting a hologram in half produces two complete smaller images, not two halves — explains CodeDNA's sliding-window robustness: even when an LLM reads only lines 50–80 of a file, it encounters Level 2 function docstrings and Level 3 call-site comments that restate the critical architectural rules at the exact point of risk.

3.2 Level 1 — Module Header

A Python-native module docstring that must be the first content in every file. It encodes the full architectural position of the file in approximately 70 tokens.

```
"""analytics/revenue.py — Monthly revenue aggrega-
tion.

deps:     payments/models.py → get_invoices_for_pe-
riod
         tenants/models.py → is_suspended
exports: monthly_revenue(year,month)->dict
used_by: api/reports.py → revenue_route
tables:  invoices(tenant_id, amount_cents, status)
         tenants(suspended_at, deleted_at)
rules:   get_invoices_for_period() returns ALL ten-
ants,
           NO suspended filter → MUST call is_sus-
pended()
         BEFORE aggregating revenue
"""
```

Using Python's native triple-quote string format (rather than a custom comment block) maximises LLM comprehension: models trained on billions of Python files apply existing pattern-matching to the docstring's structure, rather than parsing an unfamiliar protocol from scratch.

The `rules` field encodes hard constraints that apply to every edit in the file. Unlike inline annotations (function-scoped), `rules` applies globally, acting as the cell's genome: available anywhere in the file without requiring the reader to navigate to a specific function.

3.3 Level 2 — Sliding-Window Annotations

Agents frequently extract 50–100 lines from the middle of a file, bypassing the module header. Level 2 embeds navigation cues at the function level via two sub-layers.

Level 2a — Function Docstring (Google style):

```
def monthly_revenue(year: int, month: int) -> dict:
    """Aggregate paid invoices into monthly totals.

     Depends: payments.models.get_invoices_for_peri-
od
                — returns ALL invoices, NO suspended
filter.
    Rules:   MUST filter is_suspended() BEFORE sum-
ming.
    """
```

Level 2b — Call-site inline comment (last line of defence):

```
    # includes suspended tenants — filter below
    raw = get_invoices_for_period(year, month)
    total = sum(i['amount_cents'] for i in raw)
```

The call-site comment ensures that even a 10-line window centred on the dangerous call carries the critical context. This is the codon-level encoding: the smallest unit that still conveys architectural meaning.

3.4 Level 3 — Semantic Naming

Variable names encode type, shape, domain, and origin. Format: `<type>_<shape>_<domain>_<origin>`.

```
# ❌ Standard — agent must trace to understand
data  = get_users()
price = request.json["price"]

# ✅ CodeDNA — self-documenting in any 10-line win-
dow
list_dict_users_from_db  = get_users()
int_cents_price_from_req = request.json["price"]
```

4. Benchmark Design

We measure *navigation efficiency* — how many tool interactions an autonomous agent requires to correctly localize the root cause of a bug in a multi-file codebase — across two settings of increasing complexity.

4.1 Agent Implementation

We implemented an autonomous agent using Gemini 2.5 Flash (temperature 0) with three function-calling tools: `read_file(path)`, `list_files(directory)`, and `grep(pattern, directory)`. No file contents are pre-loaded. The agent runs to completion (max 20 turns) or until it stops calling tools. While temperature 0 minimises sampling variance, we note that the full agent loop (tool orchestration, API behaviour) is not strictly deterministic in the mathematical sense. Metrics: **total tool calls**, **read_file calls**, **grep calls**, **conversation turns**, and **found_correct** (boolean — whether the identified root-cause file is architecturally correct).

4.2 Benchmark 1 — 11-File Codebase

We generated two versions of an 11-file Python SaaS application (Flask server, orders, users, authentication, analytics, dashboard, payments, notifications, configuration, and database layer). **Control:** standard Python, no annotations. **CodeDNA:** identical logic, every file annotated per v0.5. Annotations describe architecture only — no bug hints.

Planted bug (B0). The user system implements *soft delete*: `delete_user()` sets `users.deleted_at = NOW()` but does not remove the row. `orders/orders.py::get_active_orders()` does not join on the users table; it returns all non-cancelled orders regardless of user deletion status, inflating the revenue dashboard.

4.3 Benchmark 2 — 105-File Enterprise Platform

We generated a 105-file enterprise marketplace platform (MarketCore) comprising 57 core modules and 48 realistic distractor files spanning `analytics`, `api`, `core`, `integrations`, `middleware`, `models`, `orders`, `payments`, `reports`, `services`, `utils`, and `workers`. The distractor files contain functions with similar names to the bug chain (e.g., `get_X_for_period`, `list_active_X`, `suspended_at`), making grep-based navigation noisy.

Control version: 1,461 lines, zero annotations. CodeDNA version: 2,437 lines, every file annotated. Three bugs were planted:

- **B1.** `analytics/revenue.py` sums invoices from suspended tenants without filtering.
- **B3.** `api/products.py` checks `user['is_admin']` instead of `payload['role'] == 'admin'`.
- **B4.** `orders/fulfillment.py` does not call `decrement_stock()` after order confirmation.

5. Results

5.1 Benchmark 1 Results (11 files)

Table 1 reports navigation metrics for the soft-delete bug (B0). Both agents correctly identified `orders/orders.py` as the root-cause file.

Table 1. *Agent navigation — B0, 11-file codebase.*

Metric	Control	CodeDNA	Δ
Total tool calls	4	3	−25%
read_file calls	3	2	−33%
grep calls	0	0	—
Conversation turns	5	4	−20%
Root cause correct	✓ Yes	✓ Yes	Equal

5.2 Benchmark 2 Results (105 files)

Table 2 reports navigation metrics across three bugs. The larger codebase amplifies the noise effect: grep queries match distractor files, forcing the Control agent to open additional files before converging.

Table 2. *Agent navigation — B1/B3/B4, 105-file enterprise codebase.*

Bug	Ctrl calls	DNA calls	Δ	Correct
B1 suspended_revenue	5	3	−40%	Ctrl ❌ · DNA ✅
B3 admin_permission	3	3	—	Both ✅
B4 fulfillment_inv	6	4	−33%	Both ✅
Total	**14**	**10**	**−29%**	1 error · 0 errors

Key result: Across both benchmarks, CodeDNA reduced total tool calls by 25–29% and eliminated incorrect root-cause identifications (0 vs. 1 in the enterprise benchmark). On B1, the Control agent identified `payments/models.py` — the location of the underlying SQL query — rather than `analytics/revenue.py` — the architecturally correct file where the filter should be applied. This distinction is architecturally significant: fixing the wrong layer would break other consumers of `get_invoices_for_period()`.

5.3 Benchmark 3 — SWE-bench Multi-Model Evaluation

To test whether CodeDNA's benefit generalises across different LLM architectures, we evaluated 3 models on 5 real Django issues from SWE-bench. Each model received identical tools (`read_file`, `list_files`, `grep`), the same system prompt, and a maximum of 30 turns. Each condition was repeated 5 times per task at T=0.1 to obtain stable mean estimates; statistical significance is evaluated with the Wilcoxon signed-rank test (one-tailed, H1: CodeDNA > Control) over the 5 per-task mean F1 pairs. The primary metric is **file localization F1**: the harmonic mean of precision (fraction of files read that are in the ground-truth patch) and recall (fraction of ground-truth files that the agent actually read). We emphasise that this metric measures *navigation accuracy* — whether the agent reads the right files — not whether it produces a correct patch. Isolating the navigation component allows us to evaluate CodeDNA's specific value proposition without conflating it with the model's code synthesis ability.

Table 3. *Multi-model file localization F1 — 5 SWE-bench tasks, 5 runs/task at T=0.1, Control vs. CodeDNA.*

Model	Ctrl F1	DNA F1	Δ F1	p-value	Tasks Won
Gemini 2.5 Flash	60%	72%	+13pp	0.040*	4 / 5
DeepSeek Chat	50%	60%	+9pp	0.11	4 / 5
Gemini 2.5 Pro	60%	69%	+9pp	0.11	3 / 5

Key result: All 3 models complete. Gemini 2.5 Flash achieves statistical significance (Wilcoxon W+=14, N=5, p=0.040, Δ=+13pp). DeepSeek Chat: Δ=+9pp, p=0.11. Gemini 2.5 Pro: Δ=+9pp, p=0.11. Positive Δ on 3/5 tasks for Pro and DeepSeek, 4/5 for Flash. Notably, task 13495 shows consistent negative Δ for both Pro (−8pp) and DeepSeek (−9pp) while Flash shows +22pp — a structural anomaly under investigation. Pro ctrl=60% matches Flash — Pro is not stronger on these navigation tasks. Per-task breakdowns in the repository.

Gemini 2.5 Flash shows the strongest benefit. One hypothesis is that Flash's more concise reasoning style is better suited to following structured annotation chains than Pro's more verbose approach. Gemini 2.5 Pro ctrl=60% matches Flash, suggesting these tasks do not favour larger models in the control condition. The three models use different provider stacks (Google native function calling vs. DeepSeek's OpenAI-compatible endpoint), which introduces some cross-provider variability not fully controlled. This does not invalidate the comparison — the annotations themselves are identical — but the observed differences reflect both CodeDNA's effect and provider-specific runtime behaviour.

5.4 Navigation Path Analysis

B1 Control (5 calls, incorrect): `list_files` → grep("revenue") → grep("invoice") → grep("suspended") → `read_file(payments/models.py)`. Three grep calls were necessary because the 105-file codebase contains 12 files with revenue-related functions; the agent used grep as a compass, opening a file mid-chain rather than following the architectural call path.

B1 CodeDNA (3 calls, correct): `list_files` → grep("revenue") → `read_file(analytics/revenue.py)`. The module header's `rules` field — *"get_invoices_for_period() returns ALL tenants, NO suspended filter → MUST call is_suspended() BEFORE aggregating"* — and the call-site comment — *"includes suspended tenants — filter below"* — gave the agent sufficient information to produce a correct diagnosis without further reads.

B4: The Control agent opened 4 files (2 greps + 2 extra reads) before converging. The CodeDNA agent opened 2 files (1 grep + 1 read): the module header stated *"MUST call decrement_stock() for each item in order['items'] before update_status()"*, eliminating the need to read `orders/models.py` and `products/catalog.py` for contextual verification.

5.5 Scaling Projection

A 29% reduction in tool calls compounds across an agentic session:

Table 3. *Projected savings across multiple sub-tasks (29% reduction assumed constant).*

Sub-tasks	Control calls	CodeDNA calls	Saved
10	50	35	15
50	250	177	73
100	500	355	145

6. Discussion

6.1 Positioning Against Related Work

Table 4 situates CodeDNA within the landscape of related approaches:

Table 4. *Problem–Solution mapping of related work and CodeDNA.*

Work	Problem exposed	CodeDNA response
SWE-bench [1]	Multi-file navigation is the bottleneck (1.96% resolution)	Reduces navigation cost via embedded dependency graph
LoCoBench [2]	Tool-usage inefficiency in long-context workflows	Directly targets tool-call reduction: −25–29%
PRDBench [3]	Over-exploration on opaque codebases	Transparent codebase structure eliminates aimless exploration
ETH Zurich [4]	External context files degrade performance −3%, cost +20%	Embedded context travels with code; no separate file to misuse
MemGPT [7]	LLMs forget prior context across turns	No memory needed: context is in the code the agent reads
RAG [6]	Retrieval latency and embedding infrastructure required	Zero retrieval: context is already at read time

6.2 Why In-Source Annotation Works

The mechanism mirrors the biological insight: when the contract governing a function is declared *at the function*, capable agents do not need to retrieve it from its original definition. The `rules` field in `analytics/revenue.py` propagated the suspension-filter contract downstream to the exact location where it was violated, allowing the CodeDNA agent to diagnose B1 correctly without reading `payments/models.py`.

The ETH Zurich finding [4] that LLM-generated context files *degrade* performance precisely motivates this approach: if external context files cause over-exploration, the solution is not better files but embedded context that arrives *with the code the agent was already going to read*. CodeDNA turns every file read into a context delivery event.

6.3 Format Design Rationale

The choice of Python-native module docstrings (rather than custom comment blocks) for Level 1 was validated empirically: models trained on Python corpora apply existing pattern-matching to the docstring's `deps:` / `exports:` / `rules:` structure. The custom `# === CODEDNA:0.5 ===` block format, while machine-parsable, required the model to process an unfamiliar syntax schema, yielding marginally longer reasoning chains before exploitation.

6.4 Annotation Overhead and the Agentic Workflow Framing

A common objection to annotation standards is the overhead of writing and maintaining them. For CodeDNA, this objection must be evaluated in context. CodeDNA is designed for *agentic code generation workflows* — environments in which an LLM agent both generates and modifies source files. In this setting, the agent itself produces the module headers and function docstrings as a natural byproduct of generation. The human developer does not annotate 500 files manually; the coding agent leaves its architectural memory in-place as it writes. The marginal cost of annotation approaches zero.

This framing also redefines what CodeDNA *is*: not a standard that asks developers to write more comments, but a convention that gives agents a structured format in which to record their own local architectural understanding. The standard converts agent-generated code from a passive output into an active, self-describing navigation graph for the next agent that reads it.

A secondary observation is that CodeDNA's structured fields (`deps:`, `rules:`, `Depends:`) do not require new syntax — they live inside Python-native docstrings. Many codebases already have docstrings; CodeDNA adds *navigation intent* to documentation that already exists. The distinction between a standard docstring and a CodeDNA docstring is not syntactic but semantic: the latter must be *machine-oriented and navigation-oriented*, not merely human-readable.

6.5 Limitations

Sample size. Benchmark 3 uses 5 runs per task at T=0.1, enabling Wilcoxon significance testing. However, N=5 tasks (all from a single Django repository) limits generalisability. Future work should expand to ≥30 tasks across multiple repositories and evaluate on the full SWE-bench suite with real-agent toolchains (Devin-style, Cursor, GPT-tools loop).

Localization vs. resolution. Benchmark 3 measures file localization F1, not end-to-end patch correctness. While localization is a necessary precondition for correct patching, it is a proxy metric. Whether CodeDNA's navigation improvements translate to higher resolution rates remains an open question for future work.

Annotation synchronization. The primary maintenance challenge is not *writing* annotations (in agentic workflows the cost is near-zero) but ensuring they remain *consistent* with the code over time. If an agent modifies a function without updating the enclosing `rules:` or `Depends:` fields, the annotation becomes misleading — potentially worse than no annotation. This shifts the research problem from production cost to *consistency enforcement*: pre-commit hooks, CI validation, and agent-side re-annotation obligations are the correct mitigations. A complementary direction is a "CodeDNA Bootstrapper" — a batch LLM pass that injects initial annotations into legacy codebases, preparing them for agentic maintenance. Future integrations should treat annotation drift as a first-class failure mode.

Multi-model evaluation. All 3 models complete. Results are consistent across models (positive Δ on 3–4/5 tasks each) but the SWE-bench subset (5 tasks, all Django) is small and all tasks come from a single repository. Task 13495 shows a consistent negative Δ for Pro and DeepSeek that is not yet explained. Cross-provider runtime differences (function-calling implementations, tool orchestration) introduce variability that is not fully controlled. The results should be replicated on a larger, more diverse task set before generalisability claims can be strengthened.

Synthetic benchmarks. Benchmarks 1 and 2 use codebases generated specifically for this evaluation, with annotations written to be accurate and complete. Real-world codebases will have incomplete or partially incorrect annotations, and performance in that setting remains untested.

Bug complexity. The planted bugs each require understanding one or two cross-file contracts. Bugs spanning five or more architectural layers may amplify the advantage further — a hypothesis that the enterprise benchmark (3 bugs, 48 distractors) provides initial evidence for.

7. Conclusion

We presented CodeDNA, a three-level in-source annotation standard whose design uses a biological analogy as a guiding intuition: just as DNA encodes a complete organism blueprint in every cell, CodeDNA ensures that every source file — and every function within it — carries enough architectural context for an LLM agent to navigate correctly without external lookup.

Our three benchmarks provide preliminary evidence that in-source architectural context can reduce navigation cost for LLM agents: (a) a 25–29% reduction in tool calls with zero incorrect root-cause identifications on custom codebases; and (b) positive improvements in SWE-bench file localization F1 across all 3 models — Gemini 2.5 Flash +13pp (60% → 72%, p=0.040), DeepSeek Chat +9pp (50% → 60%), Gemini 2.5 Pro +9pp (60% → 69%) — with positive Δ on 3–4/5 tasks per model. These results suggest that CodeDNA functions as a *navigation prior* that can benefit diverse model architectures, though the small sample size (5 tasks, single repository) limits the strength of generalisability claims.

The results are consistent with the core hypothesis: *when cross-file contracts are declared at the point of use, agents can navigate more efficiently without following each contract back to its source.* Whether this navigation improvement translates to higher end-to-end bug resolution rates is an important open question that we leave to future work with larger-scale evaluations.

The specification (v0.5), benchmark code, both codebase versions, and AI integration files are available at:

github.com/Larens94/codedna

Appendix A — Bug Description Prompts

B0 (11-file benchmark):

> *"Critical bug: orders from DELETED users keep appearing in the revenue dashboard, inflating monthly earnings. Navigate the codebase to find: (1) which files are involved? (2) why? (3) what is the fix?"*

B1 (enterprise):

> *"BUG REPORT: The monthly revenue report shows inflated figures. We suspect SUSPENDED tenants are included in the total. Navigate to find which file/function calculates monthly revenue and whether the suspension filter is applied."*

Appendix B — Codebase File Inventories

Table 5. *Key modules of the 105-file enterprise benchmark.*

Module	Files	Role
core/	5	DB, cache, auth (JWT), config, events
tenants/, users/	8	Multi-tenancy, soft-delete/suspend
orders/	6	Cart, checkout, fulfillment, models
payments/	4	Invoices, Stripe, models
analytics/	4	Revenue, usage, reports
api/	10	REST endpoints (products, orders, etc.)
distractors	48	Reports, services, utils, middleware, integrations, admin

References

[1] Jimenez, C., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.

[2] LoCoBench-Agent: *An Interactive Benchmark for LLM Agents in Long-Context Software Engineering*. arXiv, November 2025. arXiv:2511.13998.

[3] PRDBench: *Automatically Benchmarking LLM Code Agents through Agent-Driven Annotation and Evaluation*. arXiv, October 2025. arXiv:2410.XXXXX. To appear at AAMAS 2026.

[4] ETH Zurich / University of Bern (2026). *Context File Provision in LLM Coding Agents: When More Context Is Less.* arXiv:2602.11988. (LLM-generated AGENTS.md degrades task success −3%; increases cost +20%.)

[5] Chen, M., Tworek, J., Jun, H., et al. (2021). *Evaluating Large Language Models Trained on Code (HumanEval).* arXiv:2107.03374.

[6] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020. arXiv:2005.11401.

[7] Packer, C., et al. (2023). *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560.

[8] Knuth, D. E. (1984). Literate Programming. *The Computer Journal*, 27(2), 97–111.

[9] GitHub Copilot. (2024). *Custom instructions for GitHub Copilot.* docs.github.com.

[10] Anthropic. (2024). *Claude's memory: CLAUDE.md.* docs.anthropic.com.

[11] Tongyi Lingma Team (2025). *Repository-Level Code Understanding via Knowledge Graph Traversal.* Alibaba DAMO Academy preprint.